Personal Expense Analyzer — Project Report

Cover page

Project title: Personal Expense Analyzer: An LLM-Based Budgeting Assistant for Bank and Mobile-Wallet Statements

Submitted by: _____ (Name, ID)

Course: _____

Submitted to: _____

Department / Institution: _____

Date of submission: _____

Abstract

This project is a web application that analyses a person's bank or mobile-wallet statement. The user uploads a CSV or Excel file and maps its columns. All numbers are computed on the user's computer with pandas. A Large Language Model (Google Gemini) puts each transaction into a category and writes budgeting insights. Before anything is sent, card, account and phone numbers are masked. The results appear in an interactive dashboard with a budget planner, and a PDF report can be downloaded. The app gives budgeting guidance only, not financial advice.

1. Introduction

Many people do not know where their money goes each month. Payments are spread across bank cards, bKash, Nagad and cash. A monthly statement is a long list of short, unclear descriptions, so it is hard to see the full picture.

Large Language Models (LLMs) such as Google Gemini can understand short text like "POS PURCHASE FRESHBASKET" and know it is a grocery shop. This project combines an LLM with normal data analysis. The LLM reads and explains; pandas does all the counting and adding. The result is a simple personal budgeting assistant.

2. Problem statement

People face these problems when they try to manage their spending:

1. Statements show raw descriptions, not categories like "Food" or "Transport".

2. Putting hundreds of transactions into categories by hand takes hours.
3. Small repeated payments (rides, subscriptions, recharges) add up but are not noticed.
4. Unusual payments, such as a double charge, are easy to miss.
5. Most budgeting apps need a bank login or upload the full statement to a server, which is a privacy risk.

So there is a need for a free, private tool that turns a statement into clear categories, charts and budgeting tips.

3. Objectives

1. To read bank and mobile-wallet statements in CSV or Excel format, with a column-mapping step.
2. To compute income, expenses and savings locally with pandas.
3. To mask personal numbers before any data is sent to the LLM.
4. To categorise transactions with an LLM in one batched request.
5. To find recurring payments and unusual transactions.
6. To show an interactive dashboard, an editable table and a budget planner.
7. To generate a downloadable PDF report, a CSV and a JSON file.
8. To run at zero cost with the free Gemini API.

4. Key features

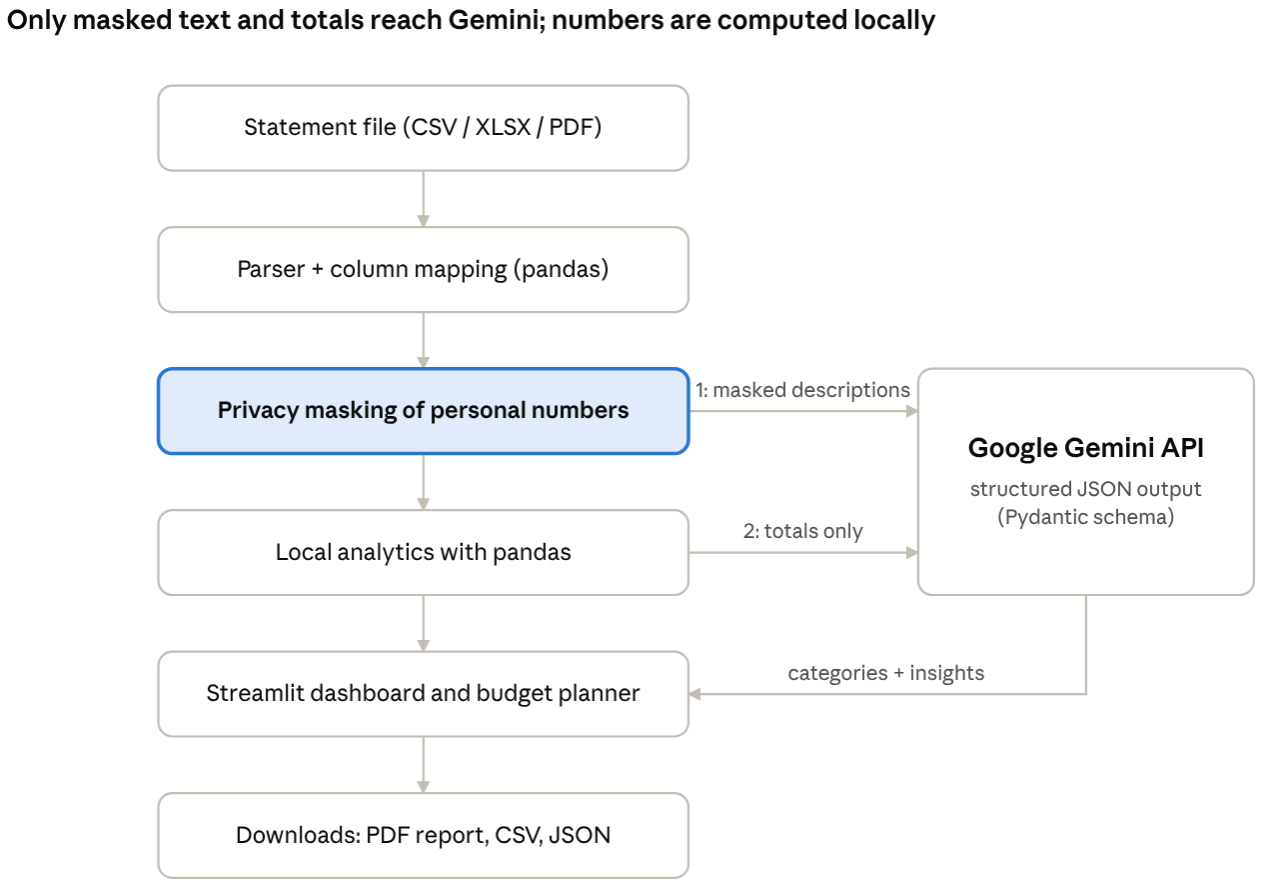
1. **Statement upload:** CSV, XLSX or text-based PDF, up to 2,000 rows.
2. **Column mapping:** choose the date, description and amount columns (one amount, debit/credit, or amount + Dr/Cr).
3. **Currency choice:** BDT (৳) by default; USD, EUR, GBP and INR also available.
4. **Privacy masking** of card, account and phone numbers, emails and reference IDs.
5. **AI categorisation** into 19 fixed categories, such as Groceries, Transport and Subscriptions.
6. **AI insights:** spending patterns, recurring payments, unusual items, saving tips and suggested budgets.
7. **KPI cards:** income, expenses, net savings and savings rate.
8. **Charts:** category donut, monthly trend and top merchants.
9. **Editable table:** the user fixes a category and all charts update.
10. **Budget planner** with progress bars showing over or under budget.
11. **Filters** by month and category.

12. **Downloads:** multi-page PDF report, categorised CSV and JSON.
13. **Demo mode** with a fake 3-month BDT statement, no API key needed.

5. System architecture

Figure 1 shows the block diagram of the system. Draw the main steps as boxes from top to bottom, with the Gemini box on the right.

Figure 1: Block diagram of the Personal Expense Analyzer



system architecture · 6 local steps, 2 Gemini requests

The file never leaves the computer. Gemini receives only masked descriptions (request 1) and summary totals (request 2).

5.1 Working procedure

1. The user uploads a statement file (or turns on Demo mode).
2. The system shows a preview and guesses the columns; the user confirms the date, description and amount columns.
3. pandas cleans the dates and amounts and keeps up to 2,000 rows.
4. Card, account and phone numbers, emails and reference IDs are masked.

5. Similar descriptions are grouped, so each merchant is sent only once.
6. The user clicks "Categorise": one request sends the masked descriptions (max 400) and the money direction to Gemini.
7. Gemini returns a merchant name and a category for each description in a fixed JSON format.
8. pandas computes the totals, monthly trend, top merchants, recurring payments and unusual transactions.
9. The user clicks "Generate insights": a second request sends only these totals to Gemini.
10. The dashboard shows the results; the user can fix categories, set budgets and download the reports.

6. Tools and technologies

Tool	Use in this project
Python 3	Main programming language
Streamlit	Web user interface, editable table (data editor)
pandas	Reading files and all calculations
openpyxl	Reading Excel (XLSX) files
Google Gemini API (google-genai)	Free LLM for categories and insights
Pydantic	Defines and checks the LLM output format
Regular expressions (re)	Masking personal numbers
Plotly	Dashboard charts (donut, bar, line, gauge)
ReportLab	PDF report with charts
pypdfium2	Reading text-based PDF statements
pytest	Automatic offline tests

7. Module description

The project is divided into small modules:

1. **Parser module** (parser.py): reads CSV, XLSX and PDF files, finds the header row, guesses the column mapping and cleans dates and amounts (including "৳ 1,200", "(500)" and Bengali digits).
2. **Privacy module** (privacy.py): masks card, account and phone numbers, emails and reference IDs, and makes a key to group similar descriptions.
3. **Analytics module** (analytics.py): computes income, expenses, savings rate, category totals, monthly trend, top merchants, recurring payments, unusual transactions and budget status. It also has offline keyword rules.
4. **Schema module** (schema.py): defines the 19 categories and the structure of the two LLM results.
5. **Prompt module** (common.py): stores the prompts, the disclaimer and the functions that build each request.
6. **Gemini module** (gemini_llm.py): lists the models, sends each request and turns errors into clear messages.
7. **Report module** (report.py): creates the multi-page PDF with charts drawn in ReportLab.
8. **App module** (app.py): the web interface with upload, mapping, filters, five views and downloads.

8. LLM and prompt design

Step 1: Privacy masking (before any request). The program replaces personal data with placeholders:

Original text	Sent to Gemini
CARD 4321-XXXX-XXXX-9876 FRESHBASKET	CARD [CARD] FRESHBASKET
Send Money to 01698765432	Send Money to [PHONE]
TRANSFER TO A/C 1234567890	TRANSFER TO [ACCOUNT]
RIDE TrxID 9F3K2L1M8A	RIDE TrxID [REF]

Similar descriptions are then grouped, so 143 sample rows become only 33 unique descriptions.

Request 1: Categorisation. The LLM acts as a transaction categoriser for Bangladeshi banks and wallets. Main rules in the prompt:

1. Choose ONE category from a fixed list of 19.

2. Use the money direction: money "in" is income or transfer in; money "out" is never income.
3. Ignore placeholders like [PHONE] and [CARD].
4. Return exactly one item per input id.

Request 2: Insights. The LLM acts as a friendly budgeting coach. It receives only totals: category totals, monthly figures, top merchants, and candidate recurring and unusual items found by rules. The rules say: quote real figures, never invent amounts, keep essentials (rent, food, health) realistic, and never suggest stocks, funds, crypto or loans.

Structured output. Both replies must follow a fixed JSON format defined with Pydantic (response schema). The program checks the JSON before using it.

One request per click, no automatic retries. This saves the free-tier quota. A full analysis uses only two requests.

Error handling. Clear messages are shown for a usage limit (HTTP 429), a wrong API key, an unavailable model and an incomplete reply.

9. User interface

The interface has three parts:

1. **Sidebar (left, dark green):** Demo mode switch, currency, Gemini API key, model, privacy note and disclaimer.
2. **Top area:** title banner, step badges (Upload, Map columns, Categorise, Insights), month and category filters.
3. **Main area:** five views chosen with a button bar.

View	Shows
Dashboard	KPI cards, category donut, monthly trend, top merchants, flagged and recurring payments
Transactions	Editable table to fix categories; live category chart
Budget planner	Budget per category, progress bars, over/under amounts
AI insights	Health score, summary, patterns, saving tips, suggested budgets
Report & export	PDF, CSV and JSON download buttons

Figure 2: Main screen of the application (draw a sketch or attach a printout).

10. Generated PDF report

The app creates an A4 PDF report of six pages. All charts are drawn with ReportLab. It contains:

1. Title, statement period, number of transactions and currency.
2. KPI cards: income, expenses, net savings, savings rate and budget health score.
3. AI summary and spending patterns.
4. Category donut chart and category table (total, per month, share).
5. Monthly income vs expenses bar chart and table.
6. Top 10 merchants bar chart.
7. Recurring payments with notes, and unusual transactions with reasons.
8. Budget plan table with progress bars.
9. Saving tips.
10. Appendix: the 30 largest payments, with masked descriptions.

Every page has a green header band and a footer with the disclaimer and the page number. The app can also download a categorised CSV and a JSON file of the full analysis.

11. Testing and results

24 automatic tests were run with pytest, plus manual tests in the browser.

Test	Result
Masking of card, phone, account, email and reference numbers	Passed
Reading the sample CSV (143 rows, 3 months)	Passed
Reading an Excel file with Bengali digits and Dr/Cr column	Passed
Income, expenses and savings rate calculations	Passed
Recurring and unusual transaction detection	Passed
No personal numbers in data sent to the LLM	Passed
Categorisation in one batched request, JSON format	Passed (simulated API)
Insights reply in JSON format	Passed (simulated API)

Test	Result
Message for usage limit (429), wrong key, wrong model	Passed (simulated API)
No automatic retry after an error	Passed (simulated API)
Demo mode, data editor and budget planner in the browser	Passed
PDF report layout (pages checked as images)	Passed

Sample result. For the fake demo statement (August to October 2026), the app shows:

Measure	Value
Income	BDT 313,312
Expenses	BDT 277,386
Net savings	BDT 35,926
Savings rate	11.5%
Largest category	Rent & Housing, 27%
Flagged items	4 rows (a large purchase, an air ticket, a double charge of 2 rows)

Note: the totals are calculated by the program from the fake statement. The categories and AI insights in Demo mode are pre-prepared demo data. A real Gemini API call has not been tested yet; the API tests used a simulated server.

12. Limitations

1. Scanned (image) PDF statements cannot be read; text PDFs work only on a best-effort basis.
2. Only the first 2,000 rows and 400 unique descriptions are analysed.
3. Recurring and unusual payments are found by simple rules, so some may be missed.
4. The LLM can put a transaction in the wrong category; the user must check and fix it.
5. Each bank uses a different file layout, so column mapping is sometimes needed.
6. Free APIs have per-minute and daily limits.
7. On the free tier, Google may use the inputs, so only masked text and totals are sent.
8. The tips are general budgeting guidance, not financial or investment advice.

13. Future work

1. Add OCR to read scanned PDF statements.
2. Add ready-made column templates for common Bangladeshi banks, bKash and Nagad.
3. Save monthly results to track spending over a full year.
4. Learn from the user's category fixes and apply them automatically next time.
5. Add savings goals and reminders for bills.
6. Support offline local models (for example through Ollama) for full privacy.
7. Add a Bengali language interface.

14. Conclusion

The Personal Expense Analyzer shows how an LLM and normal data analysis can work together. pandas computes every number on the user's computer, and Gemini only reads masked descriptions and totals. With two requests, the system gives categories, spending patterns, recurring payments, unusual items, saving tips and budgets. The dashboard, editable table, budget planner and PDF report make the results easy to use. Because it uses the free Gemini API, students and families can use it at no cost.

References

1. Streamlit documentation: docs.streamlit.io
2. pandas documentation: pandas.pydata.org
3. Google Gemini API documentation: ai.google.dev
4. Google AI Studio (free API key): aistudio.google.com
5. Pydantic documentation: docs.pydantic.dev
6. ReportLab documentation: reportlab.com
7. Plotly Python documentation: plotly.com/python